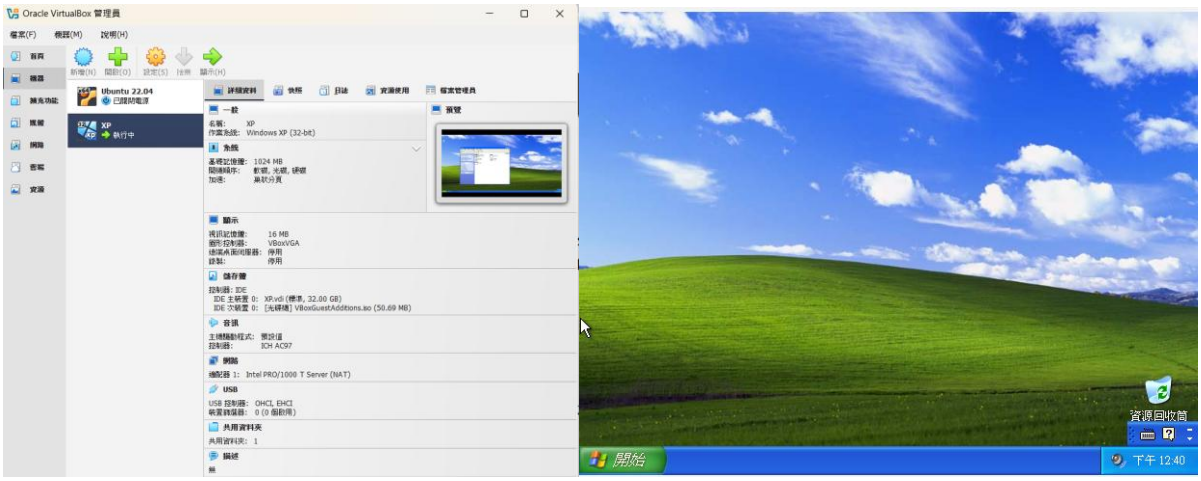
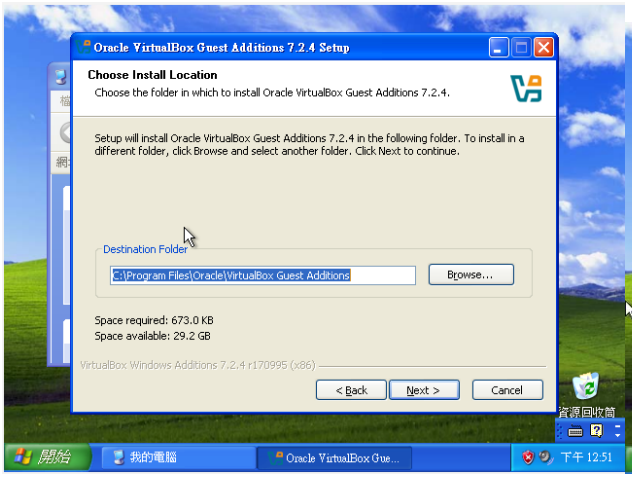
作業系統HW4

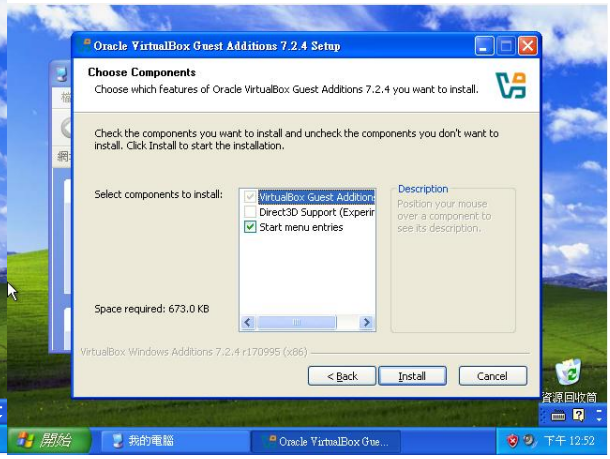
AI三 B1228005 胡樂麒

1.安裝Windows XP

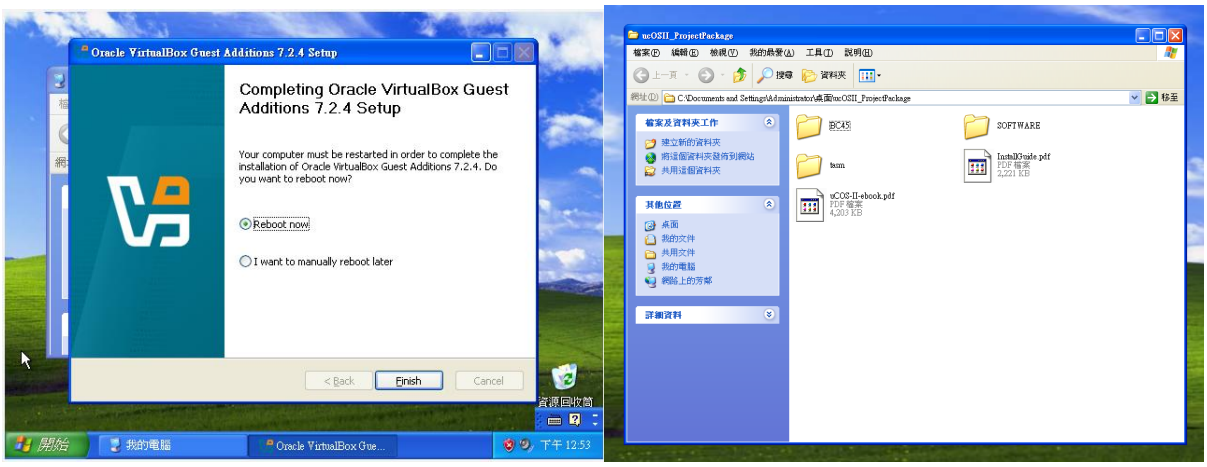


2. 安裝 “Guest Additions”

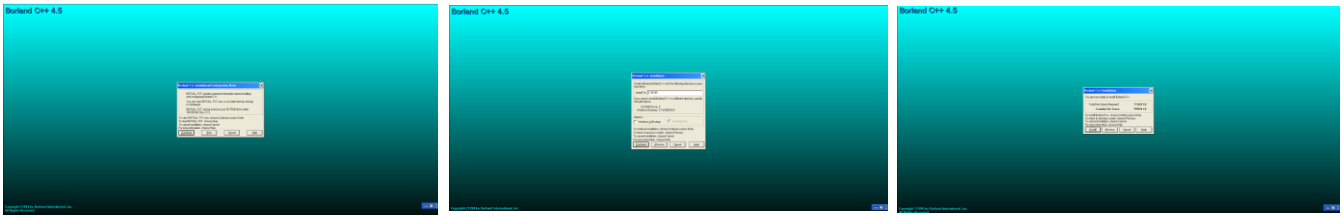


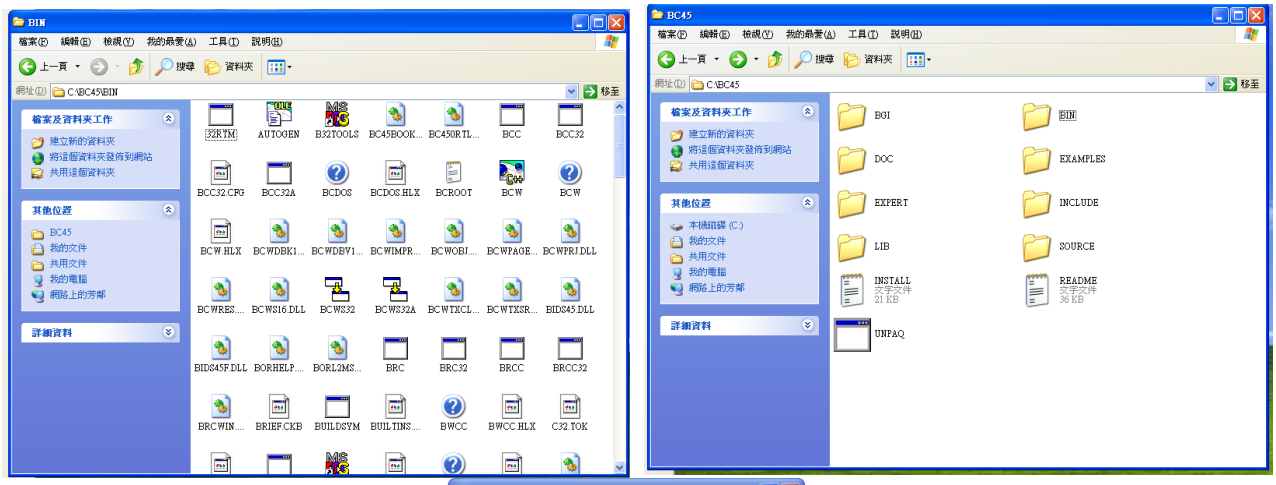


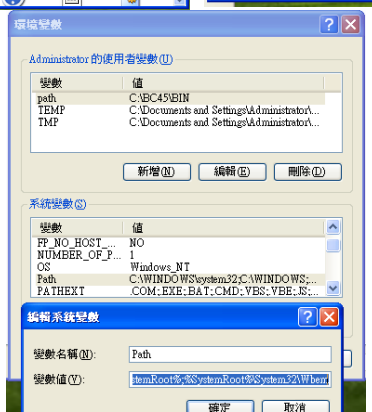
安裝成功後可以將本地檔案拖移到XP虛擬環境中



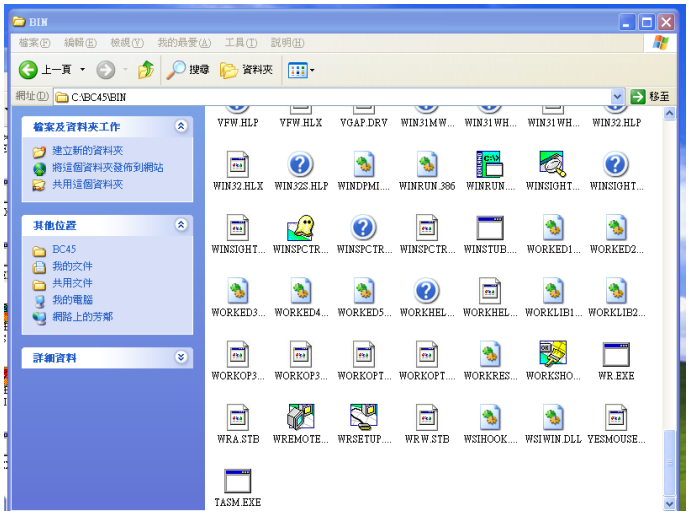
3. 安裝Borland C++與變更環境



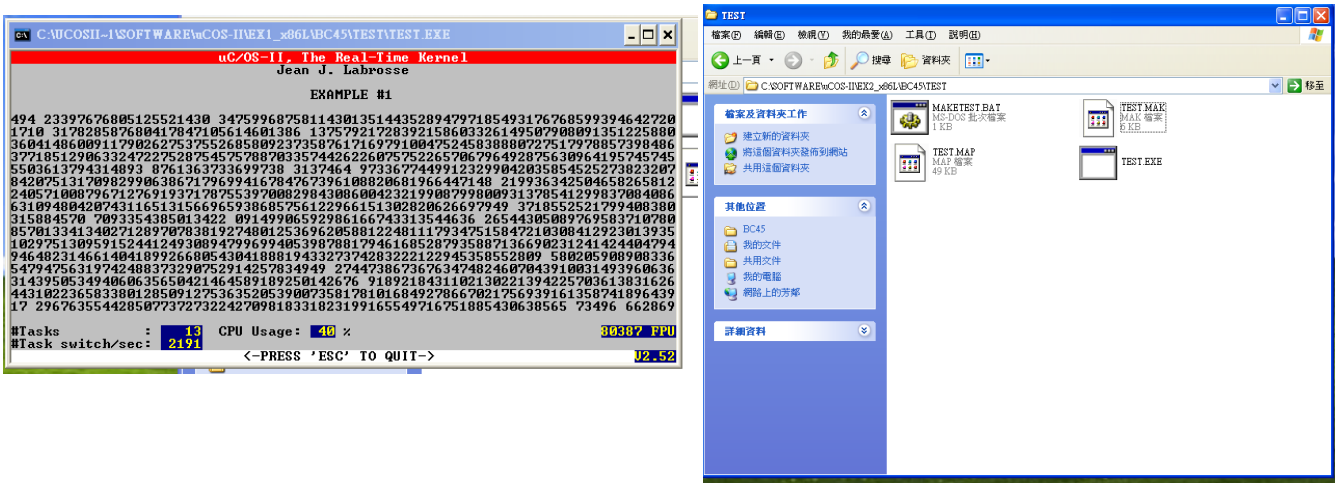




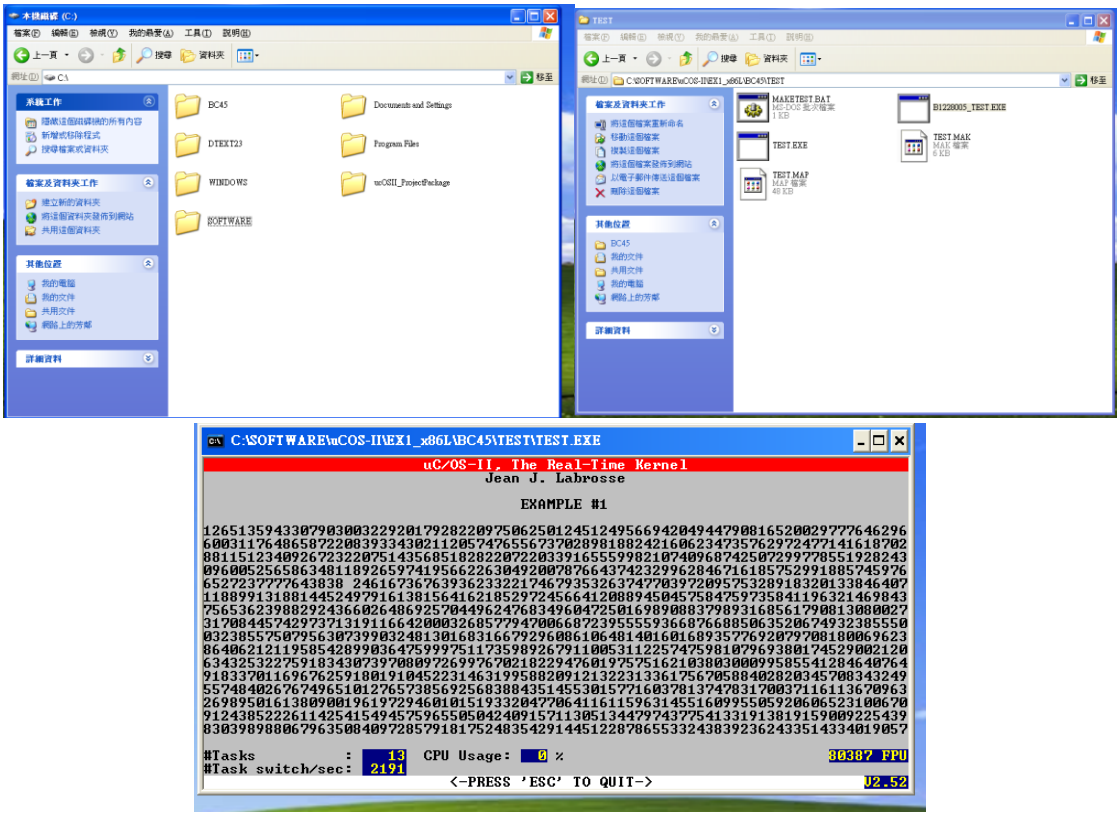
4. 將Turbo assembler解壓縮並放到 “C:\BC45\BIN”



5. 執行 µC/OS-II example code



6.將原本的TEST.TXT改名，並重新編譯出新的TEXT.TXT



7.遇到的問題

1.在安裝windows XP時，我把記憶體和硬碟空間調太大，所以我前兩次在安裝時很卡而且花非常久的時間，後來問同學安裝的設定(記憶體1024,CPU 1,硬碟32GB)才成功而且安裝很快，網路上說是因為XP無法支援現代硬體設備。

2.在編譯的時候我直接把整個資料夾放到C槽中，結果用MAKETEST.BAT一直沒辦法編譯出新的TEST.EXE回去仔細閱讀作業檔案後才發現要把SOFTWARE拉到D槽，之後就成功了。

8.參考資料:

老師的Homework 4檔案

Extra Exercise

Read the e-book of μC/OS-II

μC/OS-II 的四個範例涵蓋了 RTOS（Real-Time Operating System） 的各種核心機制，從最基本的任務切換，到 Stack 管理、Task Hooks、訊息佇列，以及最後的高頻率運算，都呈現了 RTOS 在多任務環境下的完整運作方式。Example 1 主要展示最簡單的任務排程方式：兩個任務透過 OSTimeDly() 主動讓出 CPU，輔以 PC 顯示 API 在螢幕輸出結果，直觀看到 scheduler 的切換行為。Example 2 則進一步引入 OSTaskCreateExt() 與 stack checking，透過檢查每個任務實際使用的 stack 深度，讓讀者理解“stack 太小會導致崩潰、太大則浪費 RAM”的問題，也展示 μC/OS-II 在嵌入式資源有限環境中的管理能力。

Example 3 加入更完整的 RTOS 機制，包括使用者自訂 TCB data、統計任務 Hook(OSTaskStatHook）與 Context Switch Hook（OSTaskSwHook），並建立訊息佇列(OSQCreate）來交換資料，使系統能紀錄並顯示每個任務的執行次數、耗時與 CPU 使用率，展現了 μC/OS-II 如何提供量測、除錯與監控機制，提供內部動態的重要示範。Example 4 則切換至支援浮點數的 Ix86L-FP port，建立 10 個高頻率任務進行 sin/cos 運算，在包含 idle 與 statistic task 共 13 個任務的情況下仍能維持穩定排程，展現 μC/OS-II 即便在高負載下也能確保每個任務依時序執行。

Read the source code to understand the application

INCLUDES.H

INCLUDES.H 是 μC/OS-II 範例的「標頭檔」。 它把標準 C 函式庫、μC/OS-II 核心、CPU 移植層，以及 DOS 顯示/鍵盤 I/O 函式 全部集中 include。

透過這個設計，整個測試程式只需要 include 一個檔案，就能使用 OS API、硬體支援函式、以及顯示介面。

OS_CFG.H

OS_CFG.H 是 μC/OS-II 的核心組態檔。 它決定 RTOS 在編譯時會包含哪些功能，例如 Tasks、Semaphores、Mailbox、Queue、Flag 等。

也由它控制系統可用資源的上限，例如最大任務數量、事件控制區塊、以及 Tick rate。

這種編譯客製化可以根據硬體資源與需求進行精簡或擴充。

TEST.C

TEST.C （EXAMPLE #1），使用者可以實際看到「多任務 RTOS 是如何運作的」。程式透過建立多個任務、控制顯示介面、使用 semaphore 進行同步，完整呈現 RTOS 的排程行為。

程式從 main() 開始，只做初始化、設定中斷向量與建立啟動任務（TaskStart），之後便將控制權交給 OSStart()，這也是 RTOS 與一般 C 程式最大的差異。TaskStart 在啟動後會負責初始化 Tick ISR、建立統計任務、更新畫面資訊，以及建立 10 個相同的任務（Task）。

這些 Task 會持續獲得 semaphore、隨機決定螢幕位置、顯示自己的編號並進行延遲，最終在螢幕上形成不斷跳動的數字，視覺化展示 context switch 的效果。整個程式也展示了 semaphore 如何保護共享資源（random），以及定期延遲如何讓 RTOS 平穩排程所有任務。

總體而言，TEST.C 呈現了任務建立、時間管理、資料互斥、排程器運作，以及 RTOS 主導整個系統流程的方式。這個範例讓我理解了多任務系統如何在單核心 CPU 上透過快速切換任務來達到同時處理多個任務的效果。

Browse the source code of μC/OS-II

OS_CORE.C

OS_CORE.C 是 μC/OS-II 的核心檔案，負責整個 RTOS 的初始化、任務排程、系統時間管理與事件喚醒等底層機制。程式啟動時，OSInit() 會建立 Ready List、TCB free list、Event free list，並建立 Idle Task 與 Statistic Task，使系統具備最基本的多工能力。文件中也實作了中斷進入與結束（OSIntEnter/OSIntExit）邏輯，透過記錄中斷巢狀層級來確保只有在最後一層 ISR 完成時才觸發重新排程，避免在中斷期間發生不安全的任務切換。

OS_Sched() 與 OSStart() 則負責選出最高優先權的就緒任務，並依照 Ready List 的位元表結構快速查找可執行的 Task。Idle Task 與 Statistic Task 共同提供 CPU 使用率計算，並形成系統在空閒狀態下的穩定運作基礎。Tick Handler（OSTimeTick）則負責遞減任務的延遲計數、處理超時事件，讓所有與時間相關的 API（如延遲與 Timeout）都能正確運作。

OS_CORE.C 以簡潔高效率的資料結構管理任務、透過 Tick 與 ISR 維持系統節奏，並透過一致的排程流程確保最高優先權任務隨時被執行。從這份檔案可以看到整個 RTOS 如何從初始化、排程到事件管理。